

**Project ASP.NET WebAPI and React UI
Application
Hospital Database and Appointment Manager**

Professor: Teodora Sanislav

Table of Contents:

1.Project Description	3
1.1.Key Features.....	3
2. Project Design	5
2.1. Class Diagram	5
2.2. Use Case Diagram	6
2.3. Sequence Diagrams	7
3. APIs and Frameworks	12
3.1. Backend	12
3.2 Database	19
3.3 Frontend	20
4. Version Control	26
5. Quality and Assessment	28
5.1 Application Testing Results	29
6. Workload Management	30
7. Extras	31

1. Project Description

The Hospital Database and Appointment Manager is an easy-to-use system that helps hospitals and clinics keep track of patients and manage appointments. It combines patient records, scheduling, and data reports in one place.

1.1 Key Features

The Hospital Database application has a lot of key features built into it. Some of the most important of them are the following:

Patient Management:

- **Patient Records:** Store and manage patient information, including medical history and personal details.
- **Health Records:** Easily access and update patient health records.

Appointment Scheduling:

- **Booking System:** Patients can book, reschedule, or cancel appointments online.
- **Reminders:** Send automatic reminders to patients about their appointments.
- **Resource Management:** Schedule appointments to make the best use of doctors and facilities.

Data and Reports

- **Reports:** Create reports on patient numbers, appointment trends, and resource use.

Benefits of the application

- **Efficiency:** Reduces paperwork and manual errors, freeing up staff to focus on patient care.
- **Patient Experience:** Makes it easier for patients to manage their appointments.
- **Insights:** Provides useful data to help improve hospital operations.

The Hospital Database and Appointment Manager helps healthcare facilities stay organized and provide better care to their patients.

2. Project Design

The project was thoroughly designed at the beginning of the project using 3 distinct kinds of UML diagrams.

2.1 Class Diagram

The Class Diagram was used to describe the classes of the project and to better understand the structure of the code we were going to write. We consider that a good design process is pivotal to the success of any project.

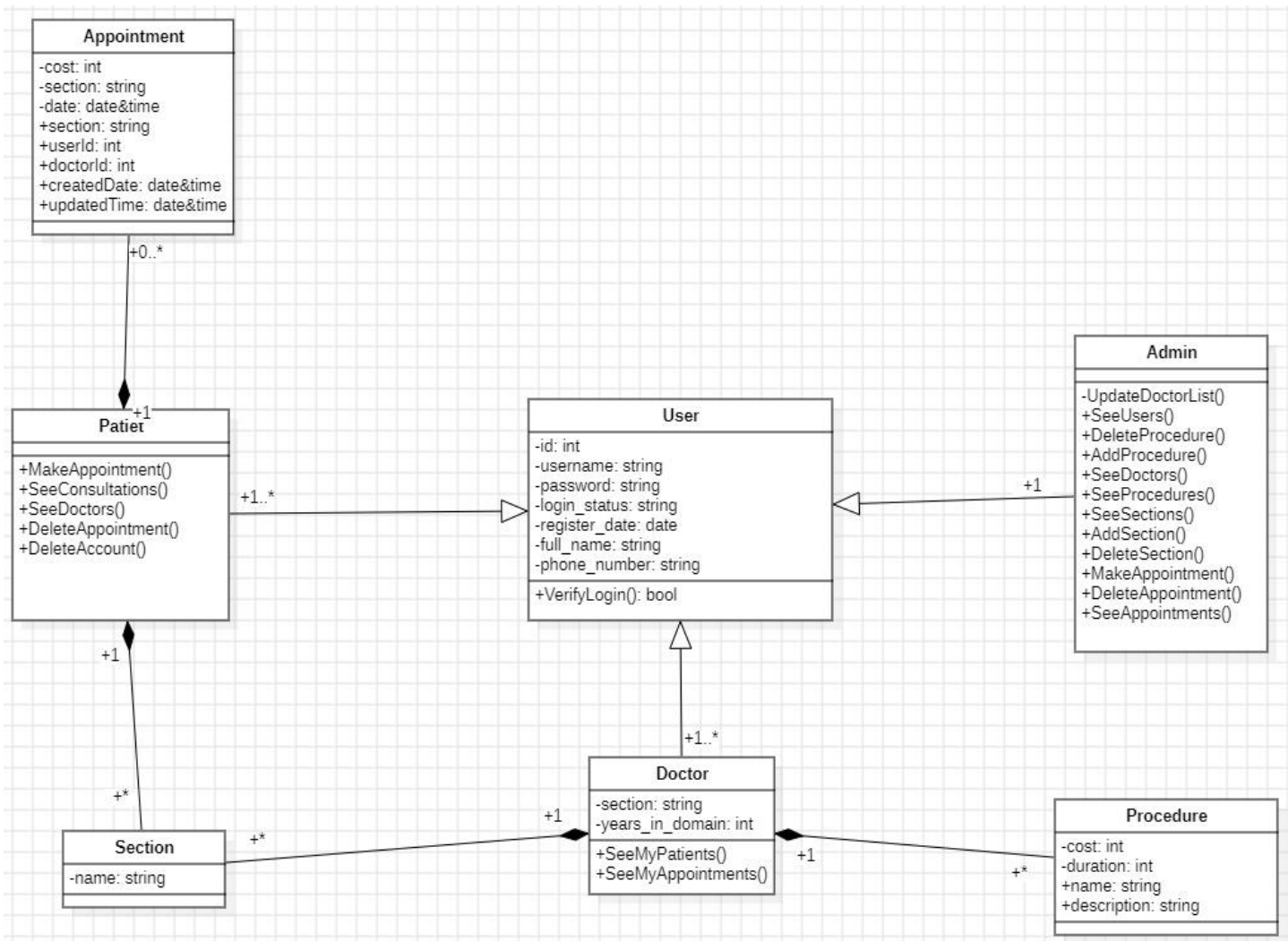


Fig 1: Class Diagram of the application

2.2 Use Case Diagram

The Use Case Diagram was used to better describe the interactions between the actors of the application and the application itself.

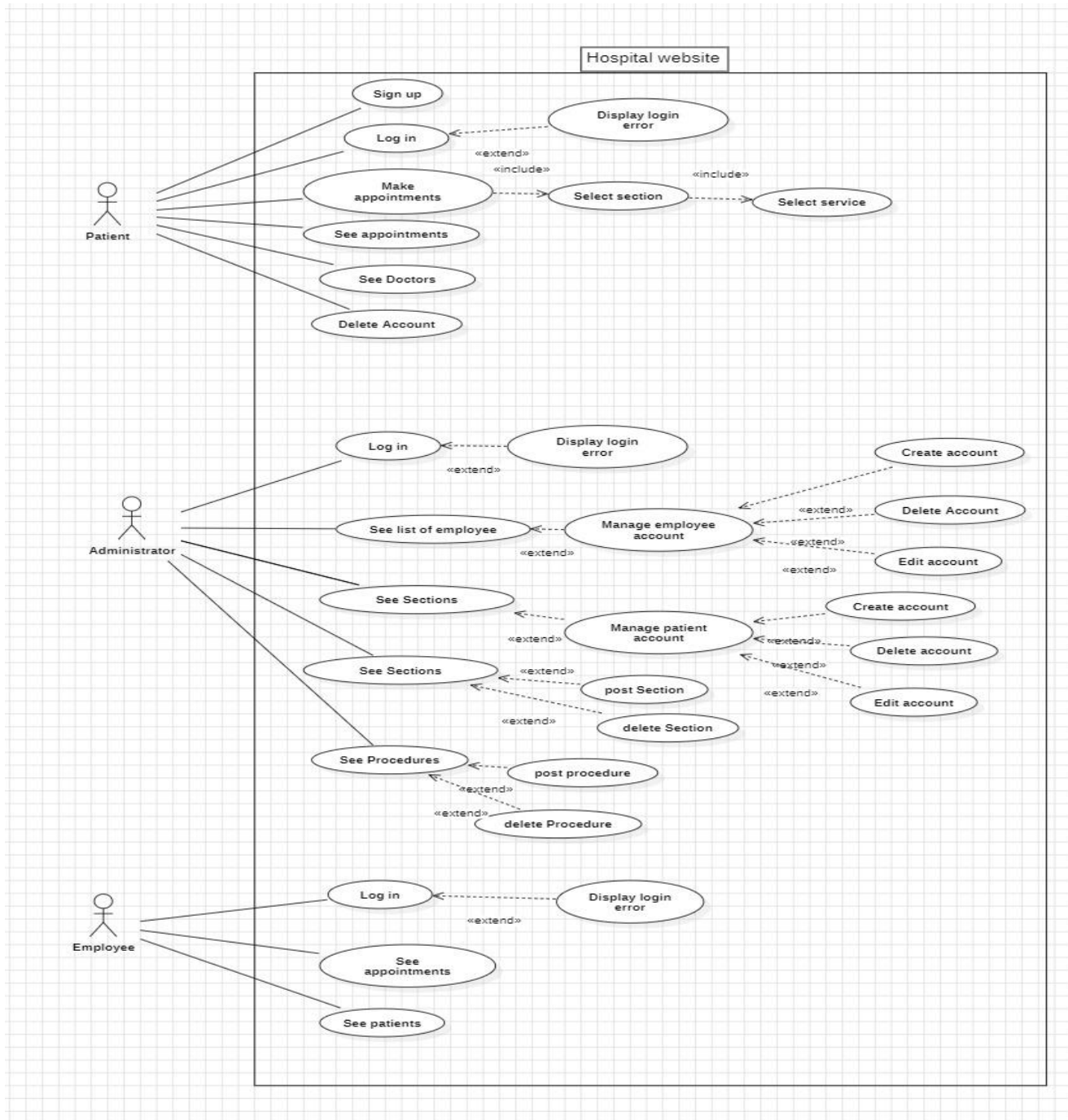


Fig 2: Use Case Diagram of the application

2.3 Sequence Diagrams

The sequence diagrams were used to outline with greater detail the sequence an actor has to use in order to achieve a desired outcome.

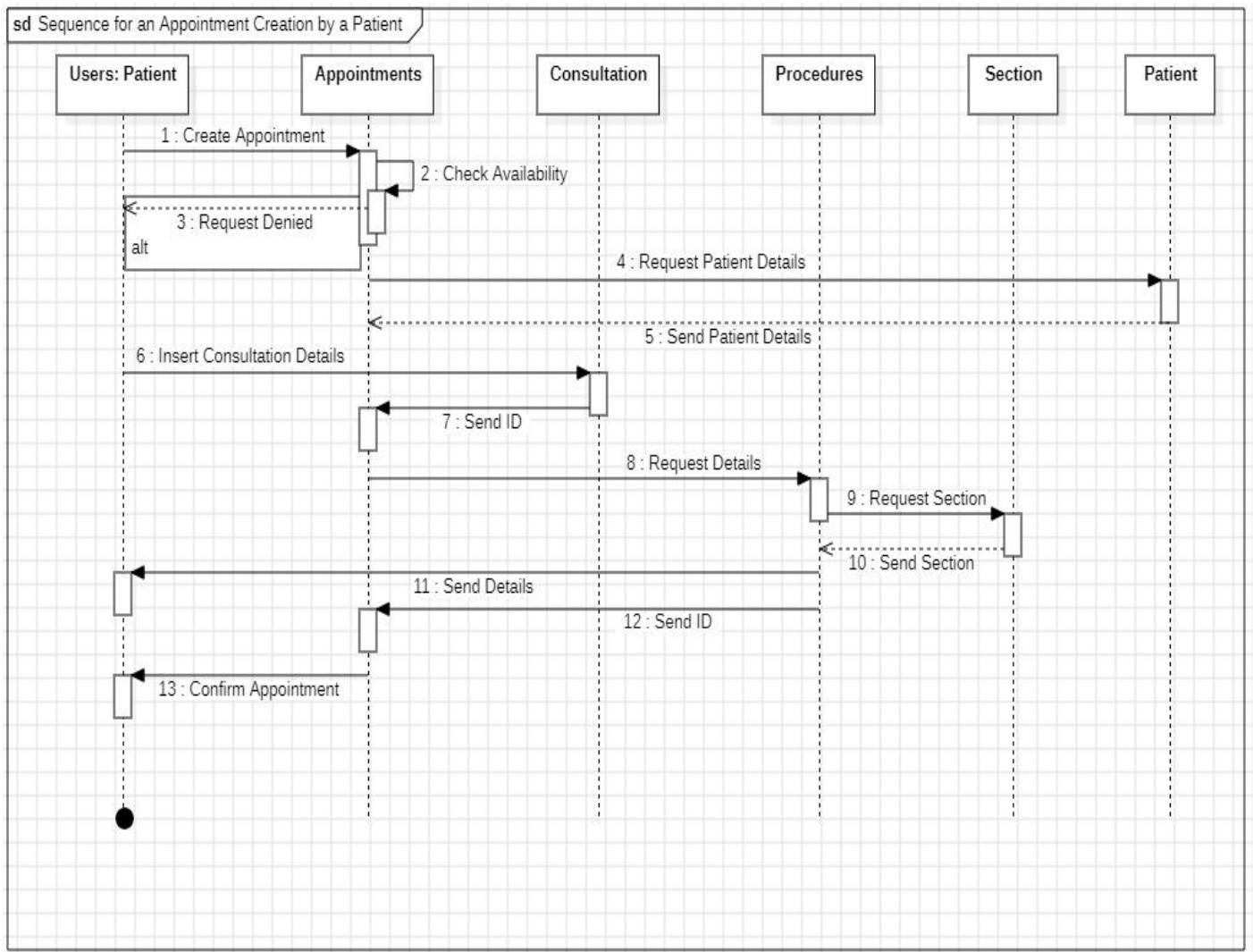


Fig 3: Appointment Creation by a Patient

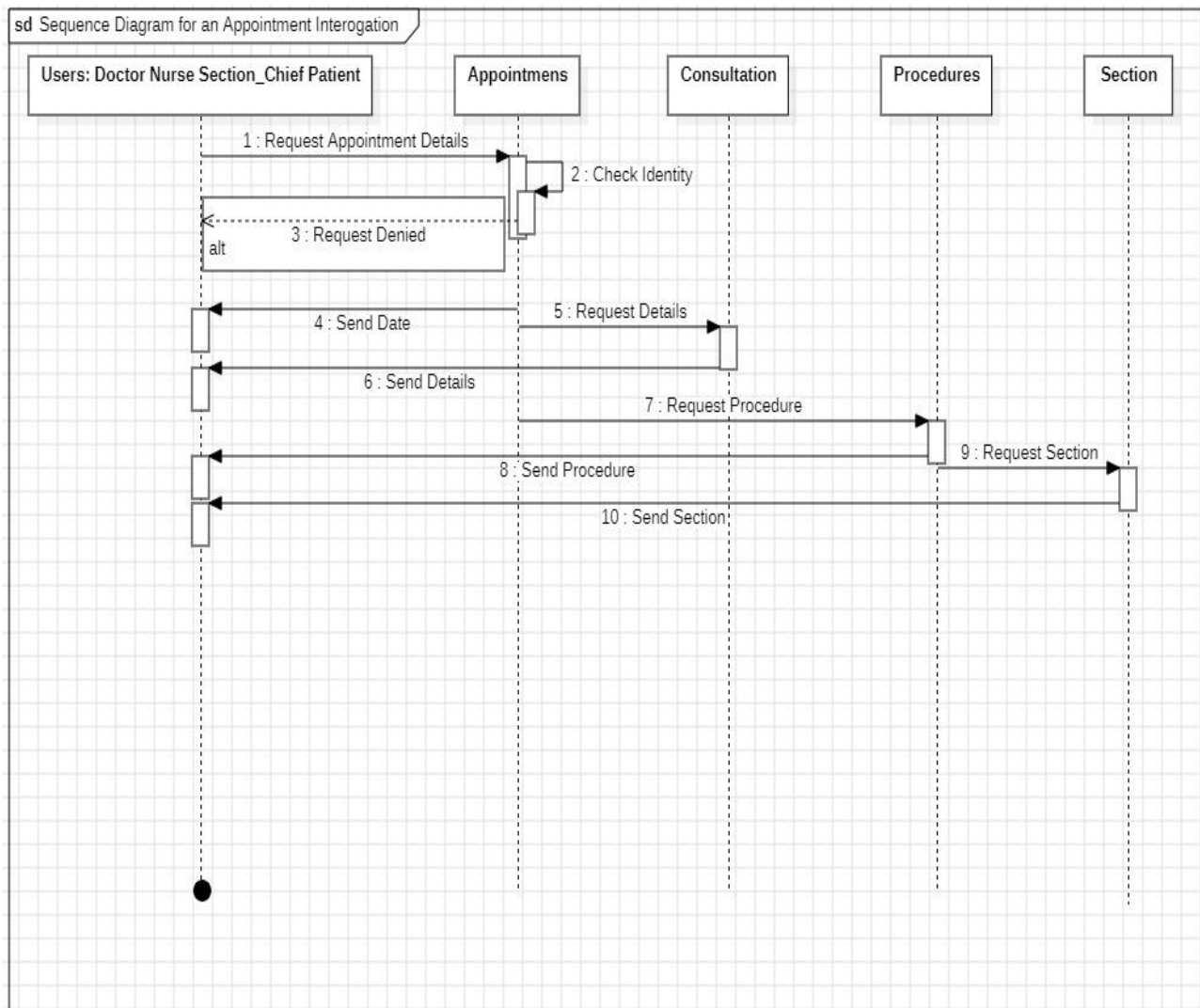


Fig 4: Sequence for an appointment creation

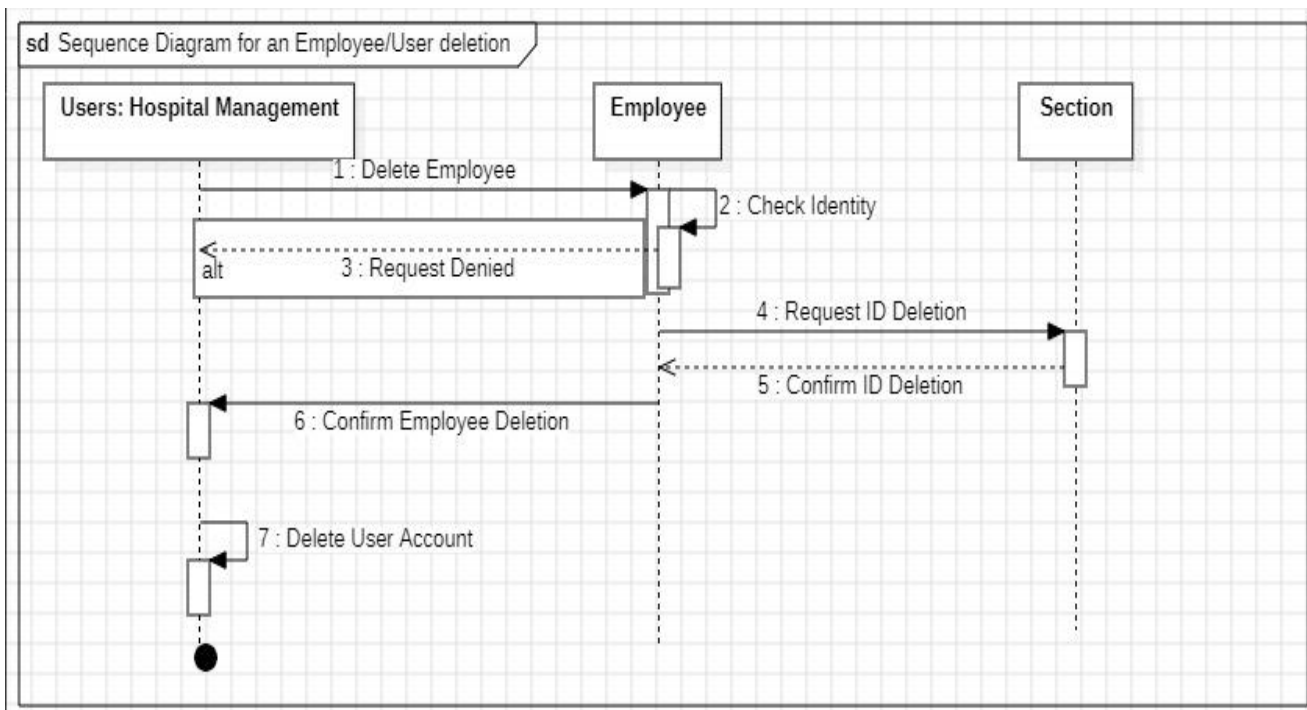


Fig 5: Sequence for User deletion

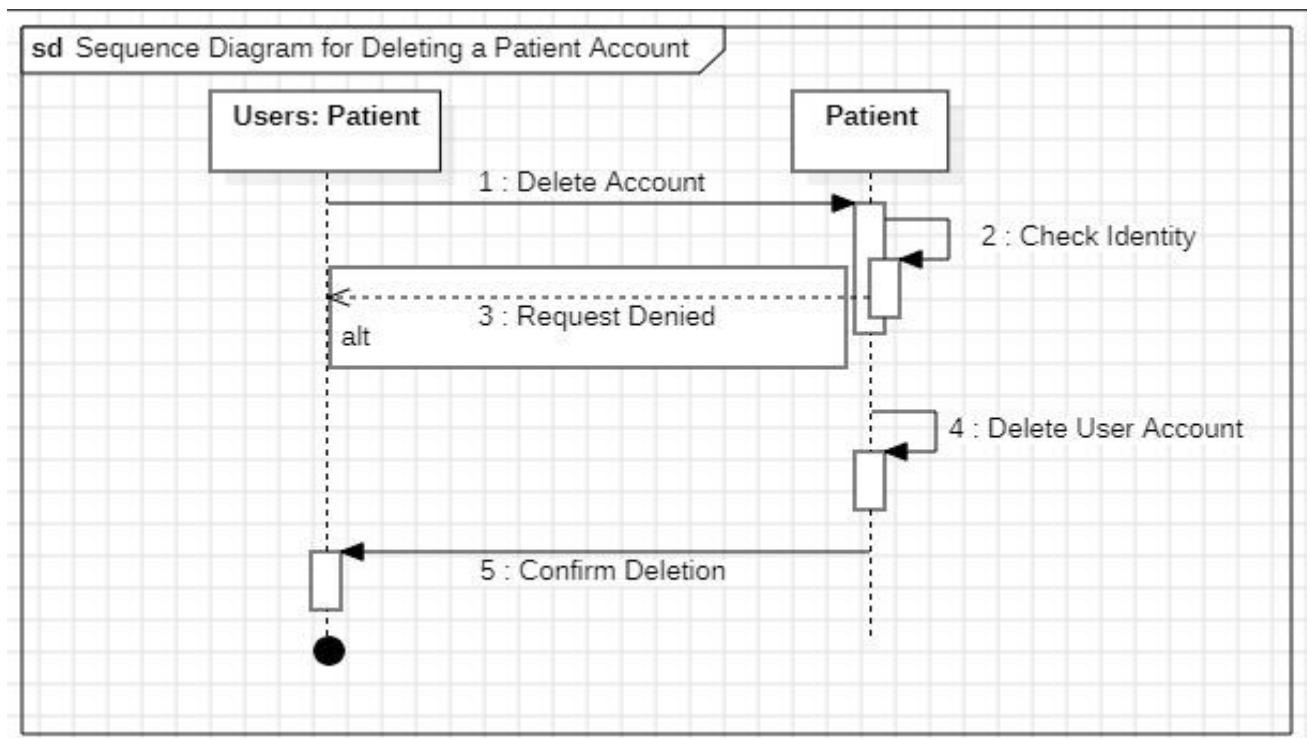


Fig 6: Sequence for the deletion of a Patient Account

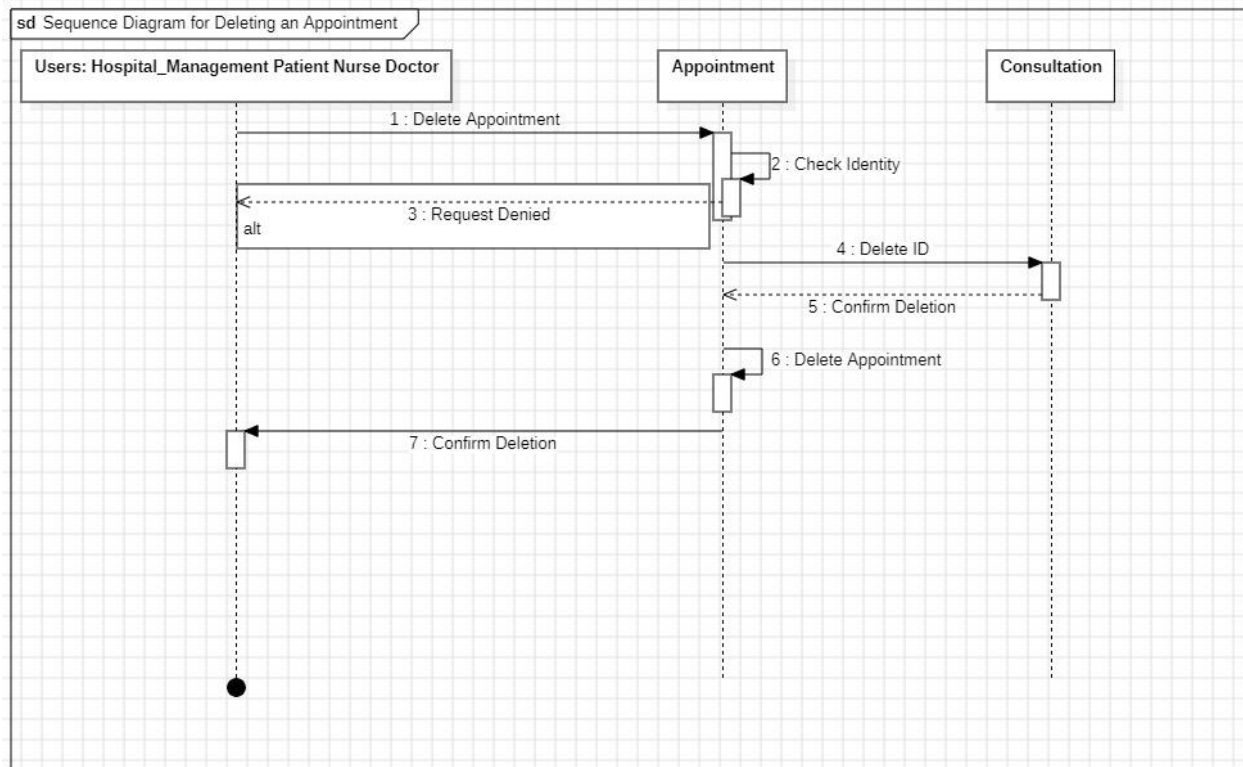


Fig 7: Sequence for an appointment deletion

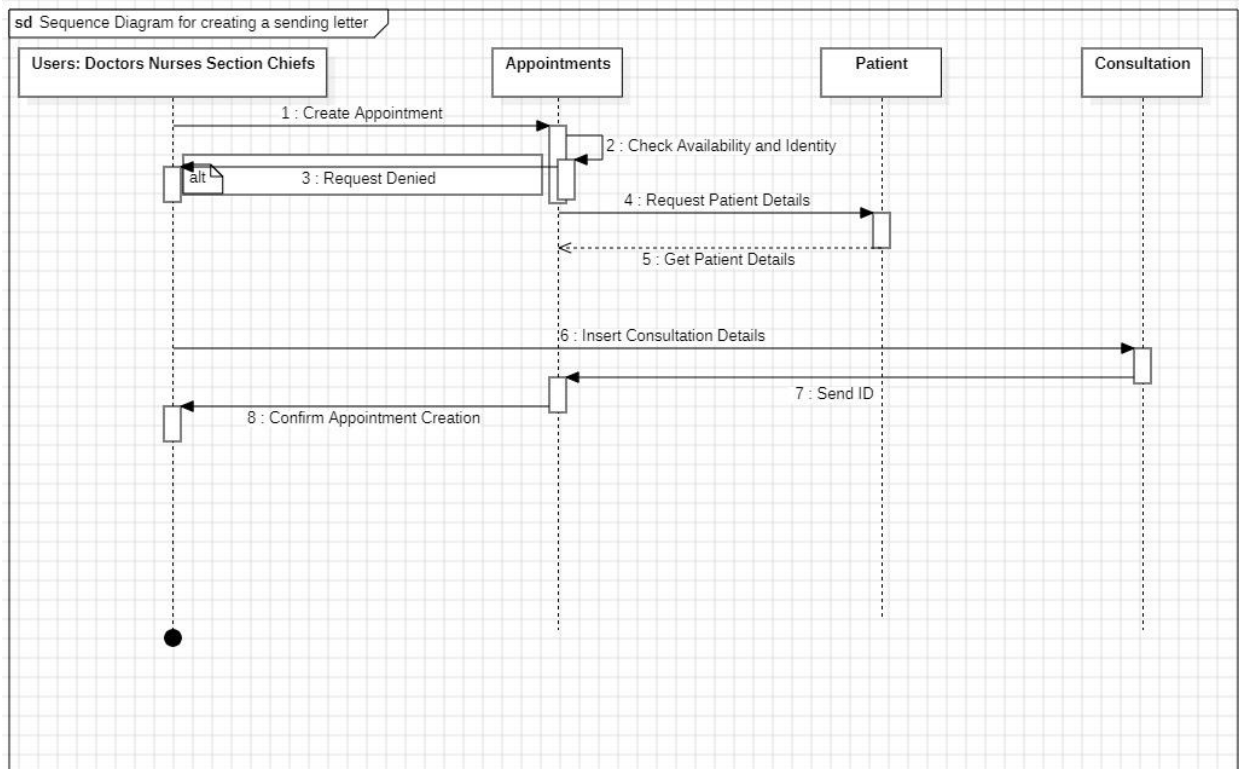


Fig 8: Sequence for creating a sending letter

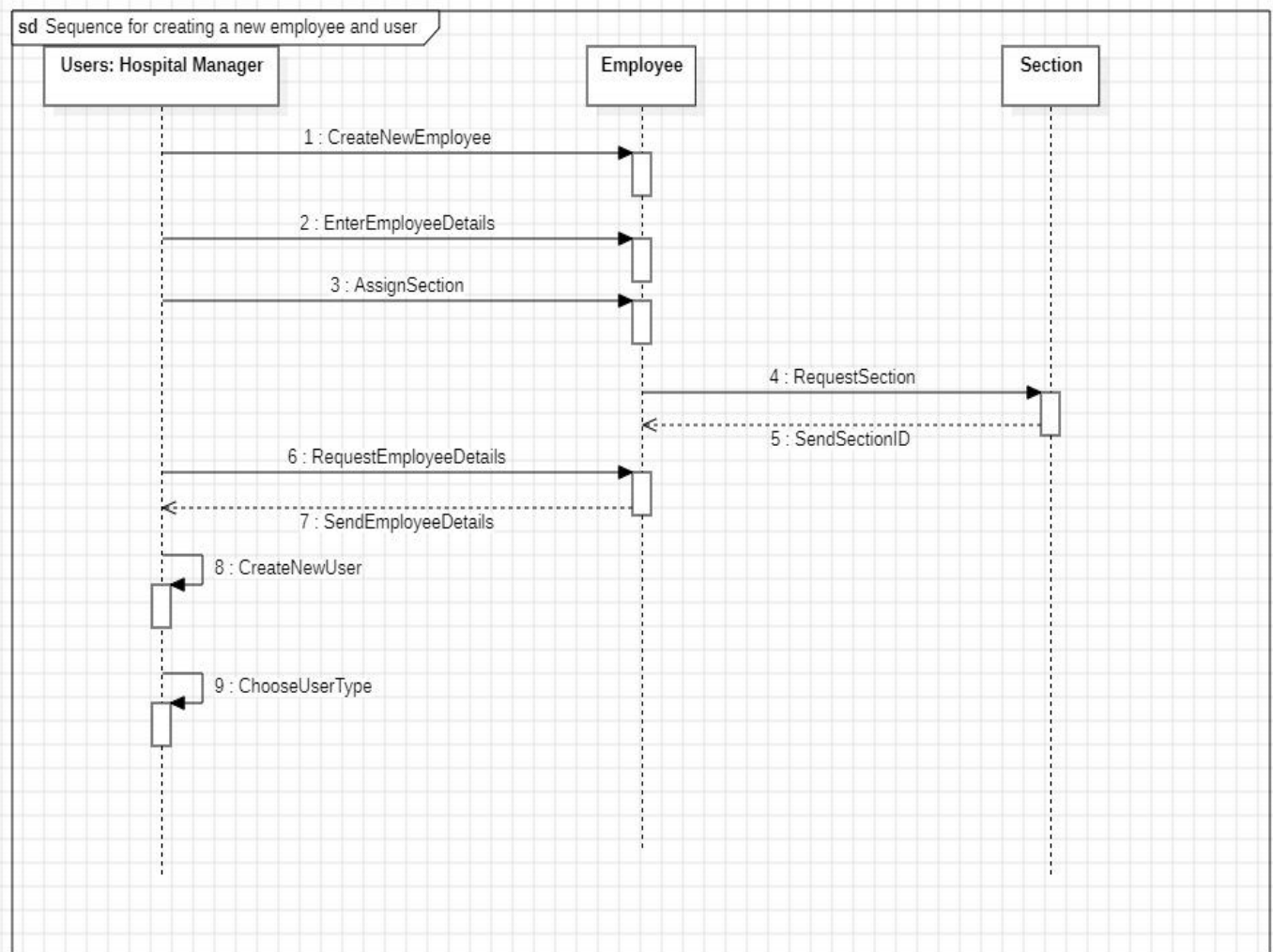


Fig 9: Sequence for creating a new User

3. APIs and Frameworks

3.1. Backend

On the backend side of the project we have opted to use the .NET API(version 8.5) Core framework for a WebApplication. Up to 9 NuGet additional packages were used in congruence with the API.

Those packages are:

- AutoMapper
- DateOnlyTimeOnly.AspNet
- DateOnlyTimeOnly.AspNet.Swashbuckle
- Microsoft.AspNetCore.Authentication.JwtBearer
- Microsoft.AspNetCore.Mvc.NewtonsoftJson
- Microsoft.EntityFrameworkCore
- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.EntityFrameworkCore.Tools
- Swashbuckle.AspNetCore

For testing the API's endpoint we have used the Swashbuckle Swagger.

3.1.1. Backend code structure

The code of the project was segmented into different parts, the most important of them all being Controllers, Data, Models and DTOs (Data Transfer Object).

We have used 4 Controllers to achieve our goals. An AppointmentController which uses different HTTPs in order to control the appointment system. A ProcedureController which is in charge of the deletion, addition and interrogation of the Hospital Procedures. A SectionController which is used to manage all the sections of the hospital.

The UserController to add, interrogate, delete and change the user settings. All the controllers use an ApplicationDbContext for connection to the database.

```
[HttpGet("{id:int}", Name = "GetAppointment")]
[ProducesResponseType(200)]
[ProducesResponseType(404)]
[ProducesResponseType(400)]
0 references
public async Task<ActionResult<APIResponse>> GetAppointment(int id)
{
    try
    {
        if (id == 0)
        {
            _response.StatusCode=HttpStatusCode.BadRequest;
            return BadRequest(_response);
        }
        var appointment = await _dbAppointment.GetAsync(u => u.Id == id);
        if (appointment == null)
        {
            _response.StatusCode = HttpStatusCode.NotFound;
            return NotFound(_response);
        }
        _response.Result = _mapper.Map<AppointmentDTO>(appointment);
        _response.StatusCode = HttpStatusCode.OK;
        return Ok(_response);
    }
    catch (Exception ex)
    {
        _response.IsSuccess = false;
        _response.ErrorMessages = new List<string>() { ex.ToString() };
    }
}
```

Fig 10: Get Appointment Endpoint

```

0 references
public async Task<ActionResult<APIResponse>> CreateProcedure([FromBody] ProcedureCreateDTO createDTO)
{
    try
    {
        if (await _dbProcedure.GetAsync(u => u.Name.ToLower() == createDTO.Name.ToLower()) != null)
        {
            ModelState.AddModelError("CustomError", "Procedure already Exists!");
            return BadRequest(ModelState);
        }
        if (createDTO == null)
        {
            return BadRequest(createDTO);
        }

        Procedure procedure = _mapper.Map<Procedure>(createDTO);

        await _dbProcedure.CreateAsync(procedure);
        _response.Result = _mapper.Map<ProcedureDTO>(procedure);
        _response.StatusCode = HttpStatusCode.Created;

        return CreatedAtRoute("GetProcedure", new { id = procedure.Name }, _response);
    }
    catch (Exception ex)
    {
        _response.IsSuccess = false;
        _response.ErrorMessages = new List<string>() { ex.ToString() };
    }
    return _response;
}

```

Fig 11: Create Procedure Endpoint

```

[ProducesResponseType(404)]
[ProducesResponseType(400)]
0 references
public async Task<ActionResult<APIResponse>> DeleteSection(string id)
{
    try
    {
        if (id == "")
        {
            return BadRequest();
        }
        var section = await _dbSection.GetAsync(u => u.Name == id);
        if (section == null)
        {
            return NotFound();
        }
        await _dbSection.RemoveAsync(section);
        _response.StatusCode = HttpStatusCode.NoContent;
        _response.IsSuccess = true;
        return Ok(_response);
    }
    catch (Exception ex)
    {
        _response.IsSuccess = false;
        _response.ErrorMessages = new List<string>() { ex.ToString() };
    }
    return _response;
}

```

Fig 12: Deleting a Section Endpoint

```

[HttpGet]
[ProducesResponseType(StatusCodes.Status204NoContent)]
0 references
public async Task<IEnumerable<User>> Get() => await _userContext.Users.ToListAsync();

[HttpGet("string")]
[ProducesResponseType(StatusCodes.Status204NoContent)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
0 references
public async Task<IActionResult> GetByRole(string role)
{
    List<User> list = new List<User>();

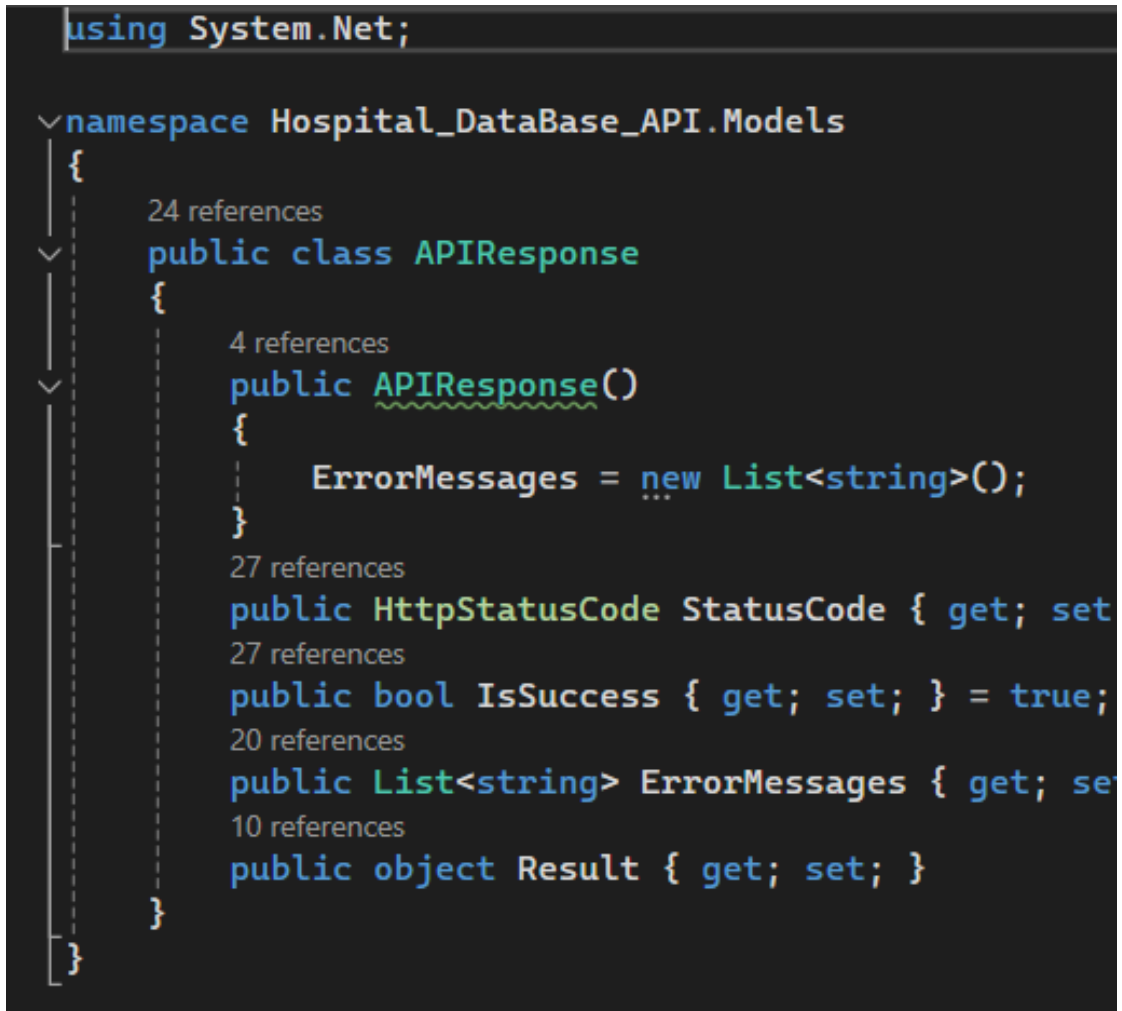
    foreach (var user in _userContext.Users )
    {
        if (user.Role == role)
            list.Add(user);
    }

    return list == null ? NotFound() : Ok(list);
}

```

Fig 13: Getting all the Users by Role Endpoint

We have used a lot of Models as the baseline for the Controllers and as the backbone of our database. We have created the models for Appointment, User, Procedure and Section alongside the DTOs for the controllers and the API Response.



```
using System.Net;

namespace Hospital_DataBase_API.Models
{
    24 references
    public class APIResponse
    {
        4 references
        public APIResponse()
        {
            ErrorMessage = new List<string>();
        }
        27 references
        public HttpStatusCode StatusCode { get; set; }
        27 references
        public bool IsSuccess { get; set; } = true;
        20 references
        public List<string> ErrorMessage { get; set; }
        10 references
        public object Result { get; set; }
    }
}
```

Fig 14: API Response Model

```

namespace Hospital_DataBase_API.Models
{
    18 references
    public class Appointment
    {
        [Key]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        4 references
        public int Id { get; set; }
        0 references
        public DateTime Date { get; set; }

        [ForeignKey("Section")]
        0 references
        public string SectionName { get; set; }
        0 references
        public Section Section { get; set; }

        [ForeignKey("Procedure")]
        0 references
        public string ProcedureName { get; set; }
        0 references
        public Procedure Procedure { get; set; }

        [ForeignKey("User")]
        0 references
        public int? UserId { get; set; }
        0 references
        public User User { get; set; }
    }
}

```

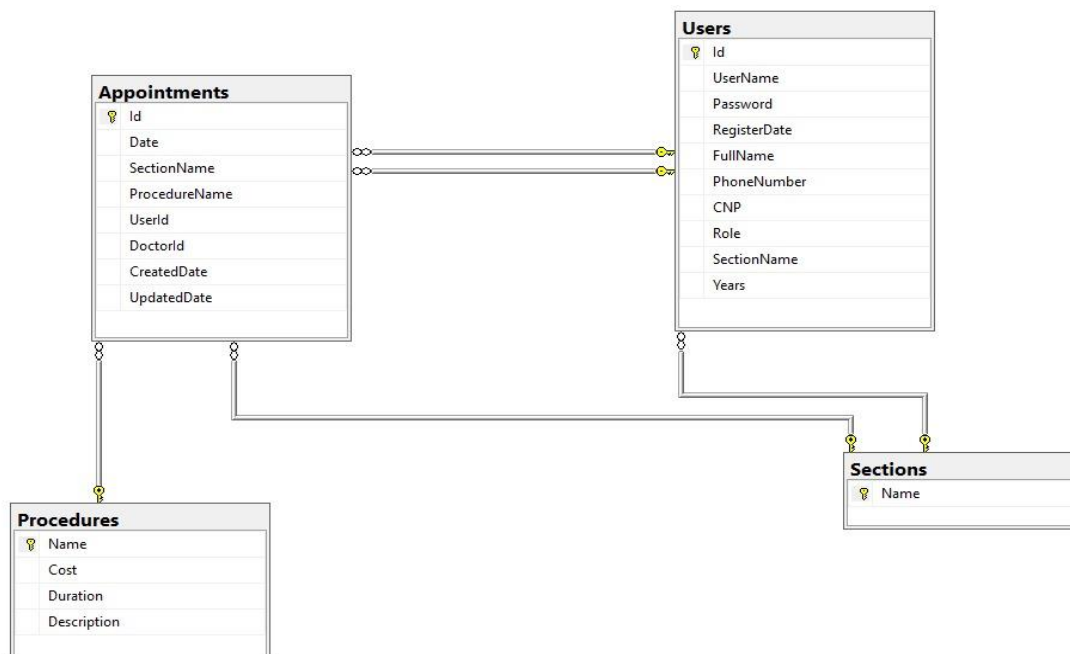
Fig 15: Appointment Model

An important milestone that we have achieved early on was the implementation of maps for a better, more efficient and optimized data and memory management system. The maps were mostly used in order to transfer data to the DTOs and for the HTTP endpoints.

The Swagger was generated every time we started the application by the Program.cs file using the “builder” statements. It played a vital role in testing and implementing the application as well as debugging and improvement of the overall application and working environment.

3.2. Database Management

Regarding the database we have opted for a “**Code First, Database After**” approach in designing our application. This means that the database is generated based on the Models using the migration operations of the Package Manager. The database uses SQL as the Query Language and SQL Server Management Studio 20 was used in order to manage it.



3.3. FrontEnd

The Hospital Database frontend application is built with React. This app makes it easier to manage and access hospital records. It provides a simple and easy-to-use interface for healthcare workers to view and update patient information quickly and securely.

Technologies Used:

- **React:**
 - A JavaScript library for building user interfaces, allowing us to create reusable components and maintain the code easily.
- **React Router:**
 - Manages navigation within the app, providing smooth transitions between different pages.
- **Redux:**
 - Helps manage the application's state, ensuring consistent data handling across all components.
- **Axios:**
 - Used for making HTTP requests to the backend, simplifying the process of fetching and sending data.

3.3.1. Composition of the Front End

The frontend is primarily composed from pages and components.

In the components folder we have all the forms for the site, like the post section form or sign up form:

```

1  import React, { useState } from "react";
2  import "../styling/loginpage.css";
3  import "../styling/form.css";
4  import { useNavigate, Form, redirect } from "react-router-dom";
5  import axios from "axios";
6  import { SERVER_BASE_URL } from "../consts";
7
8  export default function PostSectionForm() {
9      const navigate = useNavigate();
10     const [data, setData] = useState({
11         id: ""
12     });
13
14     const handleData = (e, identifier) => {
15         setData((prevData) => {
16             let newData = prevData;
17             newData[identifier] = e.target.value;
18             return newData;
19         });
20     };
21
22     return (
23         <>
24             <Form
25                 method="POST"
26                 action="/admin-account/admin-see-sections/post-section"
27                 className="form-container"
28             >
29                 <h1>Post Section</h1>
30                 <div className="input-container">
31                     <label htmlFor="name"></label>
32                     <input
33                         placeholder="name"
34                         type="text"
35                         className="short-input"
36                         id="name"
37                         defaultValue={data["name"]}
38                         name="name"
39                         onChange={(e) => handleData(e, "name")}
40                     />
41                 </div>
42                 <button className="submit-btn">Post Section</button>
43             </Form>
44         </>
45     );
46 }

```

Fig16. Post Section Form

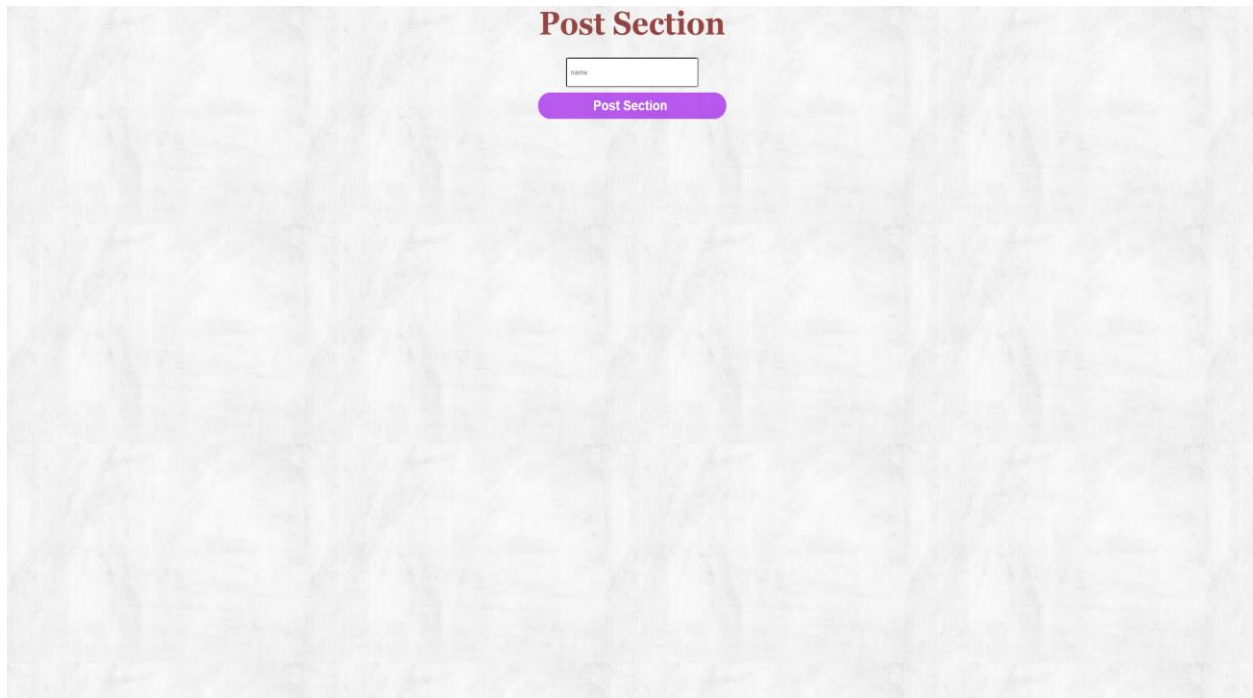


Fig 17: Post Section

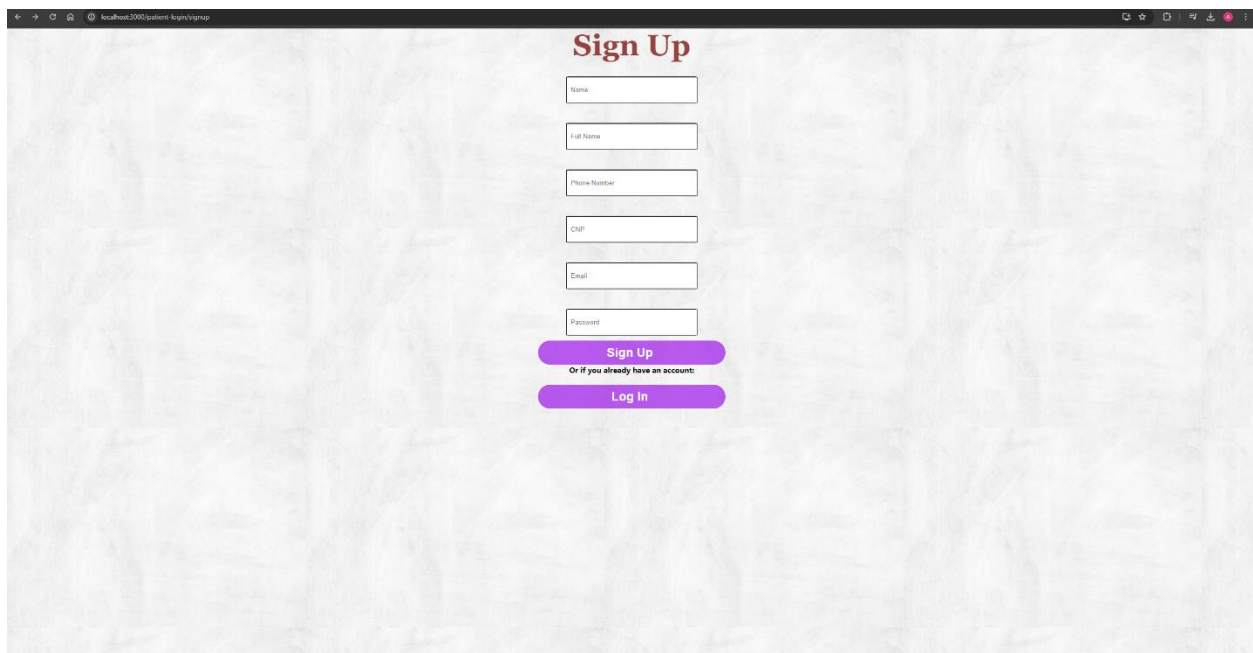


Fig 18: Sign Up Page



Fig 19: Home Page

This is the Home page, where each character icon, from left to right, represents a specific user type: Patient, Doctor, or Administrator. Each user type has distinct permissions and functionalities within the site.

For example the Patient can make an appointment, see the doctors, see his appointments, delete appointments, delete account, and see the available sections, when he is logged in into an account.

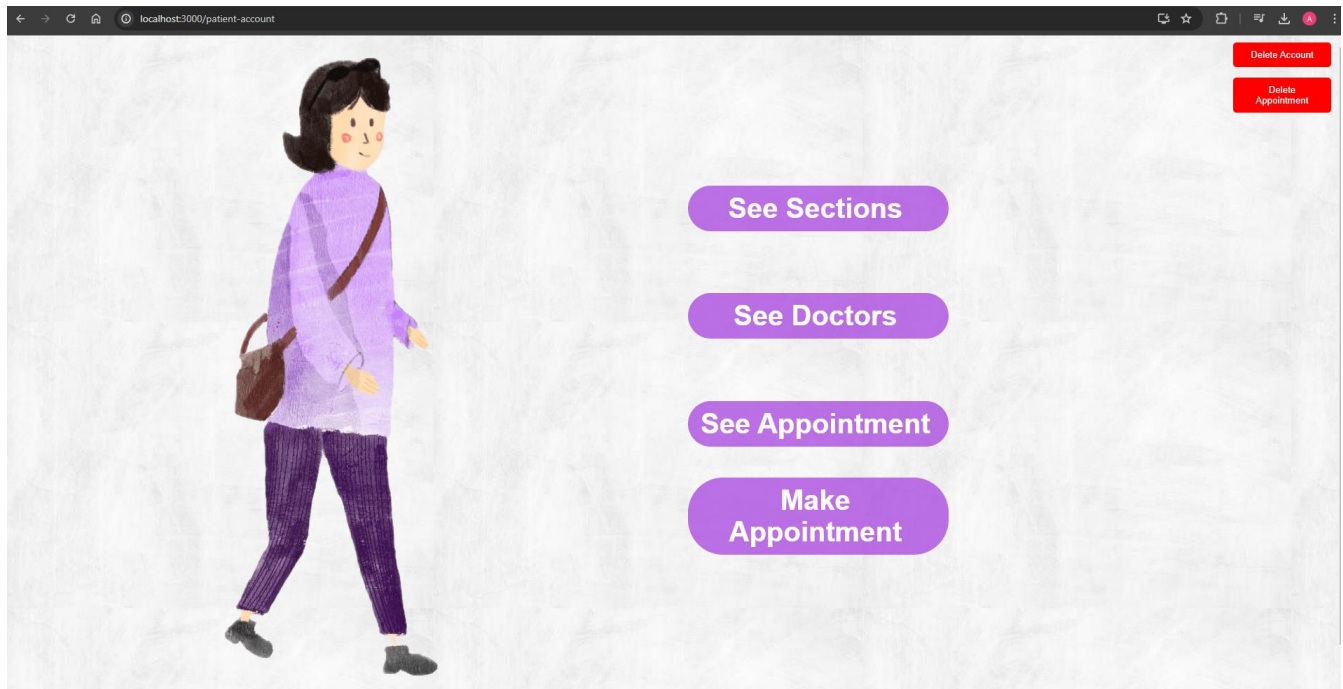
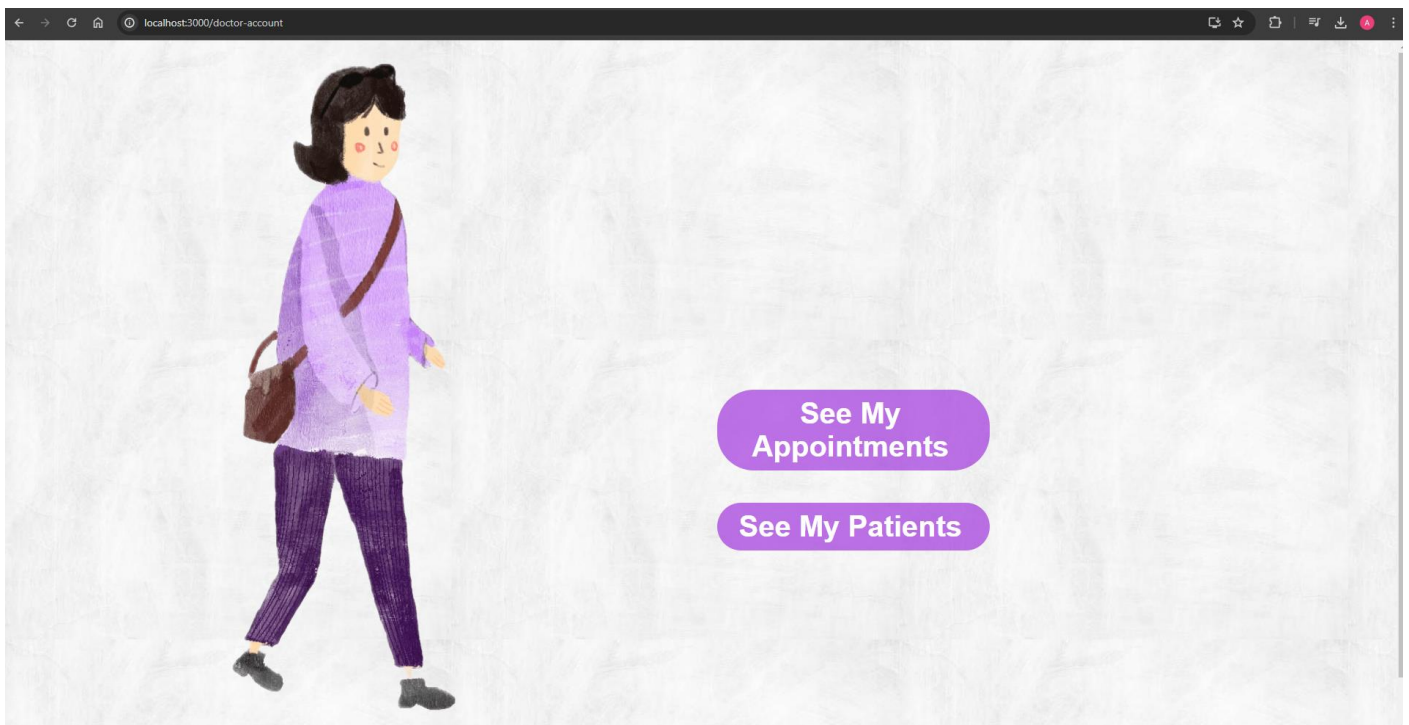


Fig 20: Patient Account

If you log in as a doctor you will have the next options: See your patients and See your appointments



And lastly if you log in as an administrator you can see and update the users see and modify the sections same with the procedures and make, delete and see all appointments

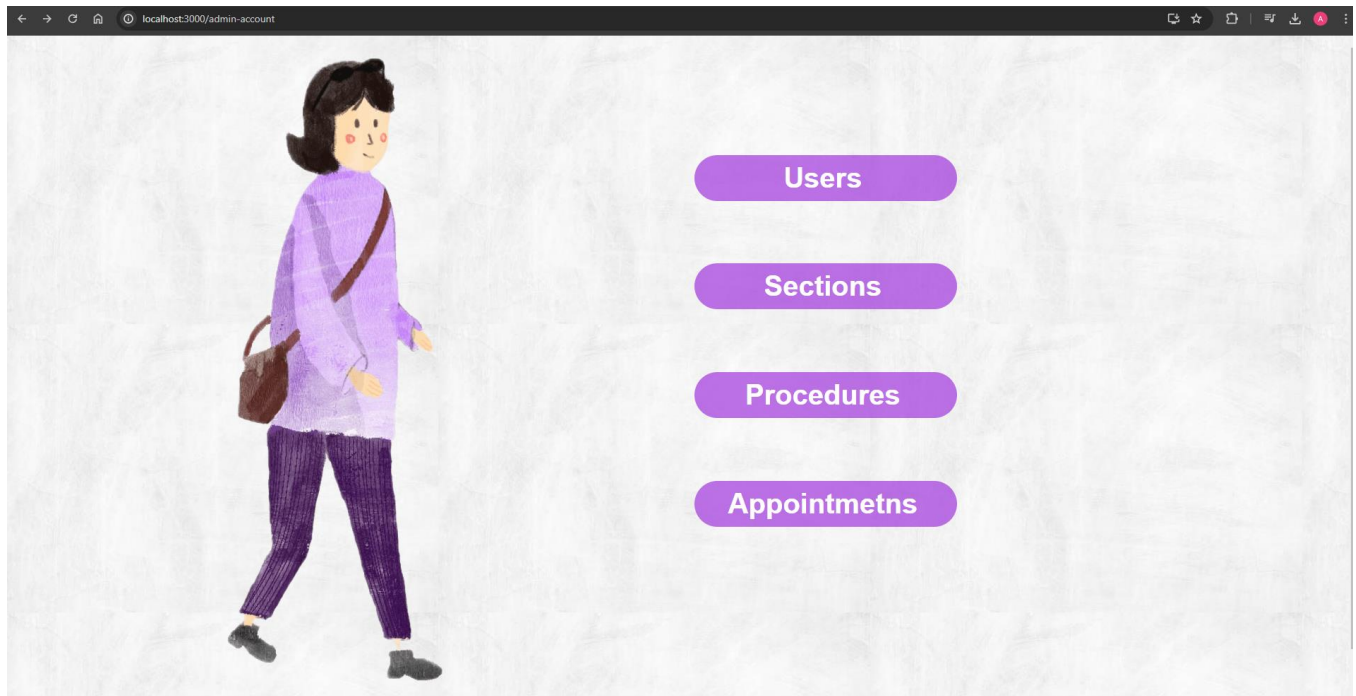


Fig 21: Admin Account

4. Version Control

As a version control tool we decided to use GitHub with the Bash terminal. We worked on branches with “develop” being the main product branch and we were following the classic branch conventions:

- feature/<taskID>_<taskDescription> for adding new features to the application.
- bugfix/<taskID>_<taskDescription> for fixing the inevitable bugs

We opted for this approach due to the fact that it was way easier to just make a new branch and work on it without the fear of breaking the whole application and the fact that we can all work at the same time on the same source code.

Another advantage of using Git for source control is the accessibility options that come with it, from a remote repository that can be accessed by anyone to the ability to reset “develop” to previous states.

In the below picture we can see the source tree with the branches that were created and then later merged into “develop”. As we can observe the tree looks extremely clean due to the fact that we chose this workflow.

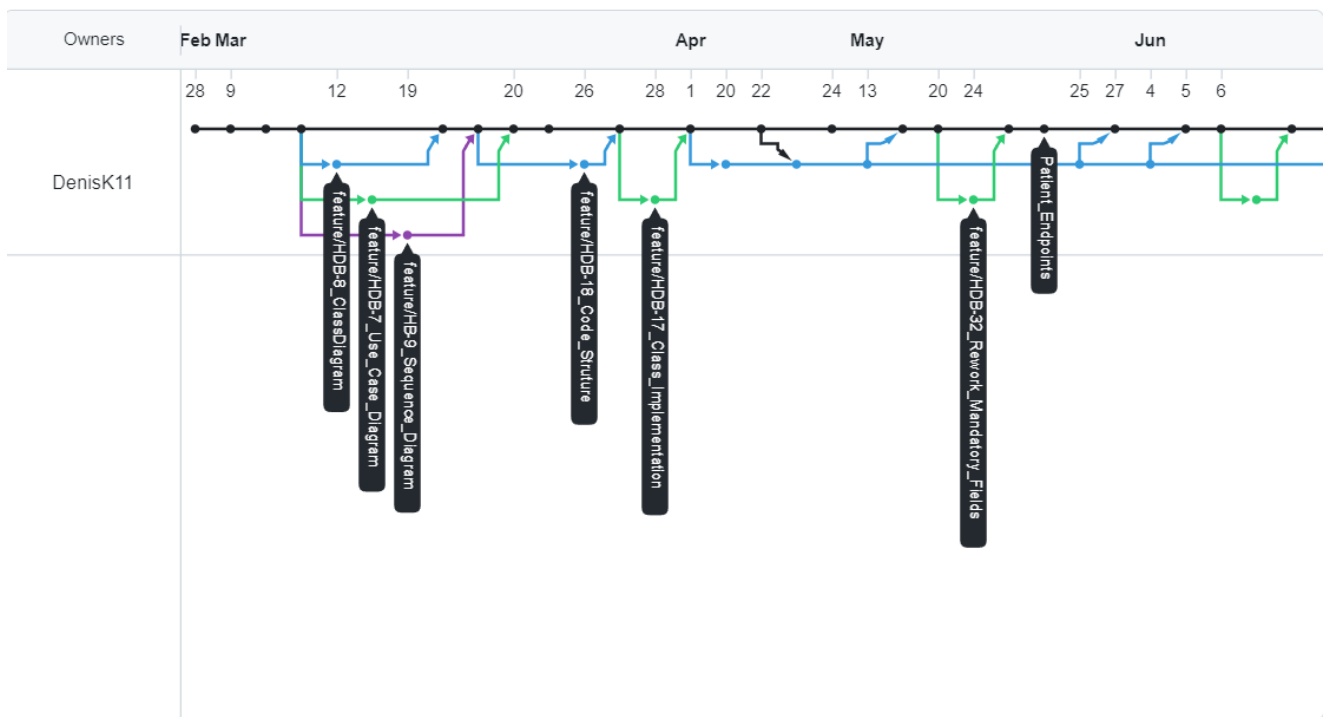


Fig 22: Source tree

Branch	Updated	Check status	Behind	Ahead	Pull request
develop	3 hours ago			Default	...
Your branches					
Branch	Updated	Check status	Behind	Ahead	Pull request
feature/HDB-34_End_point_creation_for_the_new_pages	8 hours ago		4	1	...
feature/HDB-18_Code_Structure	3 months ago		29	0	#4 ...
feature/HB-9_Sequence_Diagram	3 months ago		33	0	#2 ...
Active branches					
Branch	Updated	Check status	Behind	Ahead	Pull request
feature/HDB-34_End_point_creation_for_the_new_pages	8 hours ago		4	1	...
frontend	8 hours ago		16	0	#10 ...
Patient_Endpoints	yesterday		13	0	...
feature/HDB-32_Rework_Mandatory_Fields	2 weeks ago		15	0	#7 ...
feature/HDB-17_Class_Implementation	3 months ago		24	0	#5 ...

Fig 23: A sneak peek into the branches

5. Quality and Assessment

Testing the application plays a crucial role in ensuring its usability and quality. We tested our app using the Swagger. By the end of the testing, all the requirements have been satisfied and all the tests have been passed.

We have made a Requirement Module in the project so we can all understand what the application has to do by the end of the project and to ensure its quality and scalability.

Requirements

Requirement ID	Requirement
HDBR_01	The application shall be able to save the data of the users in a database.
HDBR_02	The application shall be able to recall the data from a database.
HDBR_03	The application shall be able to create, recall, modify and delete appointments created by the user in question.
HDBR_04	The application shall have functional "Log in" page for all the types of users.
HDBR_05	The application shall have a functional "Sign up" page for the relevant users.
HDBR_06	The application shall allow the relevant users to change any relevant data regarding themselves.
HDBR_07	The application shall allow the relevant users to delete any information regarding themselves.

Fig 24: Some of the written requirements

5.1. Application Testing Results

In the image below we can observe the application testing results of the project based on the above mentioned requirements. The application tests results can be found in the archive of the project.

Case#	Description	Expected Result	Actual result	Passed/Fail
1	Pacient tries to log in	Login succesfull	Login succesfull	Passed
2	User tries to create new pacient account	Sign Up succesfull	Sign Up succesfull	Passed
3	Pacient tries to create new appointment	Appointment succesfull made	Appointment succesfull made	Passed
4	Pacient tries to delete account	Account succesfully deleted	Account succesfully deleted	Passed
5	Pacient tries to view his appointments	Appoints succesfully displayed	Appoints succesfully displayed	Passed
6	Employee tries to log in	Login succesfull	Login succesfull	Passed
7	Employee tries to view his appointments	Appoints succesfully displayed	Appoints succesfully displayed	Passed
8	Administrator tries to log in	Login succesfull	Login succesfull	Passed
9	Administrator tries to view all employees	Employees succesfully displayed	Employees succesfully displayed	Passed
10	Administrator tries to view all sections	Sections succesfully displayed	Sections succesfully displayed	Passed
11	Administrator tries to create a new section	Section succesfully created	Section succesfully created	Passed
12	Administrator tries to delete a sections	Sections succesfully deleted	Sections succesfully deleted	Passed
13	Administrator tries to create a new employee account	Employee account succesfully created	Employee account succesfully created	Passed
14	Administrator tries to delete an employee account	Employee account succesfully deleted	Employee account succesfully deleted	Passed
15	Administrator tries to create a new procedure	Procedure succesfully created	Procedure succesfully created	Passed
16	Administrator tries to delete a procedure	Procedure succesfully deleted	Procedure succesfully deleted	Passed

Fig 25: Application Tests Results

6. Workload Management

One of the hardest part of living in the current age is time management, especially this semester. In order to have a better odds of achieving good results in every aspect of our academic performance we needed to adopt a task tracking system.

We opted for Jira made by Atlassian because is easy to use, learn and understand. Jira is highly customizable and can be linked with the GitHub repository for easier access to the pull requests, task and branch tracking as well as application releases.

Using Jira we created a Scrum project. The role of ScrumMaster was fulfilled by the team leader which made, updated and kept track of the tasks as well as the Epic system that was in place with Scrum project type.

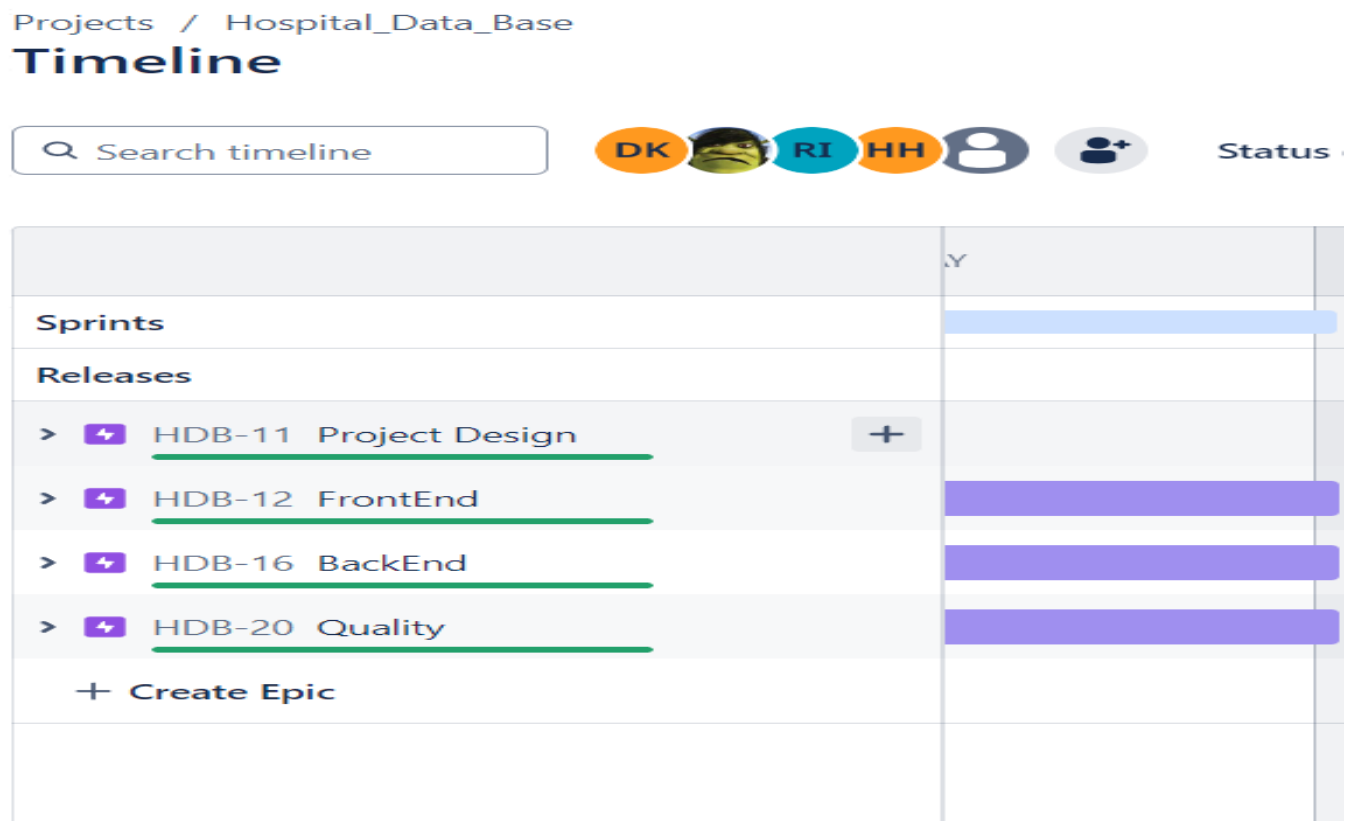


Fig 26: Task Tracking

7. Extras

Team:

Kuth Denis-Ioan	Team Leader / Developer
Horea Haragis	Developer
Rad Emilian Antonio	Developer
Iles Ioan Rares	Tester / Developer

Links:

<https://universityprojectutcn.atlassian.net/jira/software/projects/HDB/boards/3>

[https://github.com/DenisK11/Hospital Database](https://github.com/DenisK11/Hospital_Database)